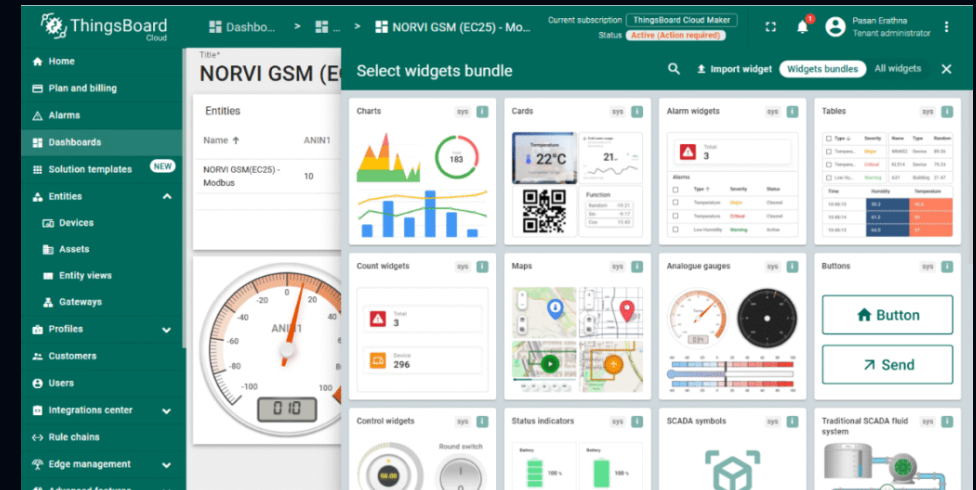


PROJECT III: IOT APPLICATION

Embedded Design with Raspberry Pi

UDP Communication, Buildroot Linux & ThingsBoard Integration



PROFESSOR
Mariano Ruiz

PRESENTED BY
Covaliov Mocenco Alexandru
Ungureanu Radu Ionut
Tita Andrei Dragos

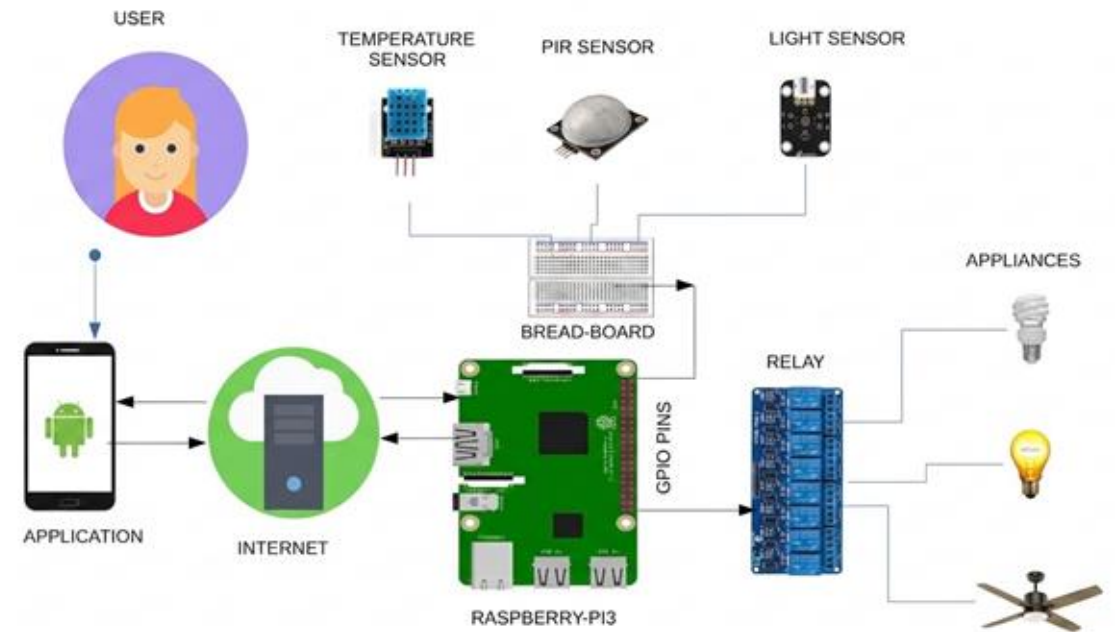
DATE
May 2026

Context & IoT Vision

ITU-Y.2060 Definition

"A global infrastructure enabling advanced services by interconnecting physical and virtual things based on interoperable ICT."

Ubiquitous Computing: Computing present everywhere, anytime, driven by Context Awareness.



Project Core Objectives

- ✓ **Implement Client-Server:** Reliable UDP communication between RPi and virtualized Ubuntu.
- ✓ **Periodic Sampling:** Acquire data from MPU6050 and TCS34725 sensors every 1s.
- ✓ **Cloud Gateway:** Establish a bridge between RPi and Cloud dashboard.

CORE STACK

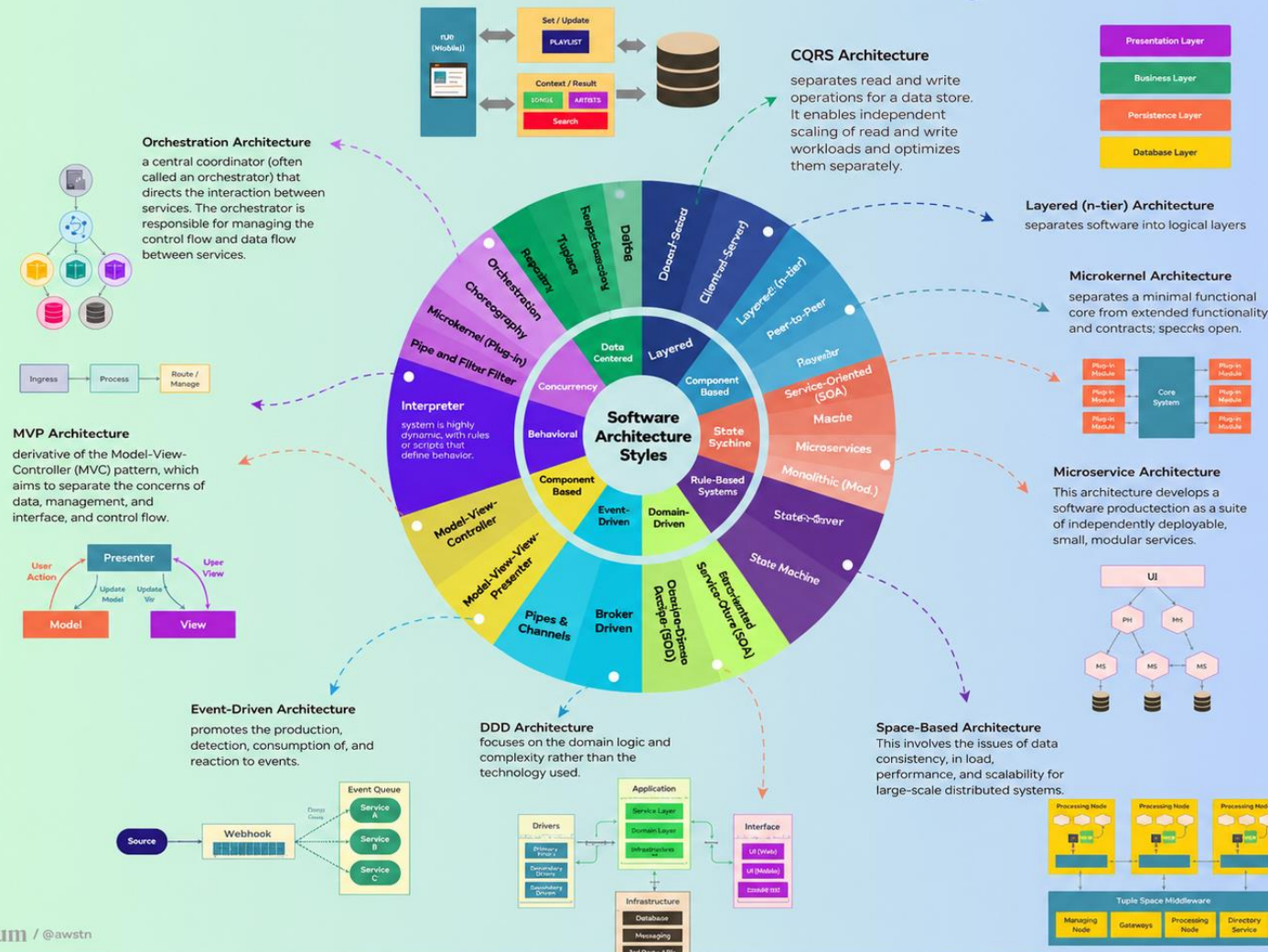
Buildroot Embedded Linux

C/C++ POSIX Sockets

VMware Ubuntu 24.04.01

System Topology (LAN)

Software Architecture Styles



Medium / @awstn



RPI (UDP Linux)



Host Ubuntu (UDP Linux)



ThingsBoard (MQTT)

UDP Client/Server Lifecycle



```
// Client Setup (Raspberry Pi)
int sockfd;
struct sockaddr_in server_addr;

// Create socket UDP
sockfd = socket(AF_INET, SOCK_DGRAM, 0);

// Configuration Ubuntu
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = inet_addr(SERVER_IP);

//Communication
sendto(sockfd, &packet, sizeof(packet), 0,
        (const struct sockaddr *)&server_addr, sizeof(server_addr));
```

Hello World Handshake: Validates connectivity before sensor data transmission.

No Hard-coding: Parameters (IP/Port) are handled dynamically for code flexibility.

Buildroot Kernel: Custom Linux environment deployed on Raspberry Pi.

Sensor Fusion & Sampling

PITCH

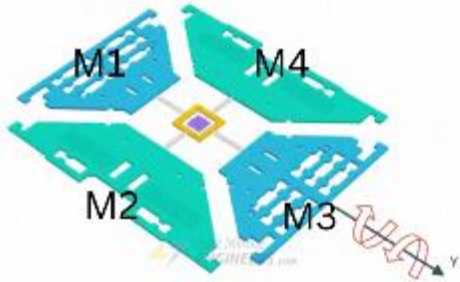


Fig.1

ACCELEROMETER

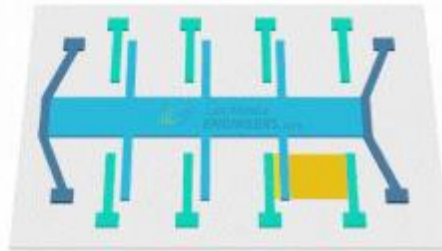


Fig.4

ROLL



Fig.2

YAW

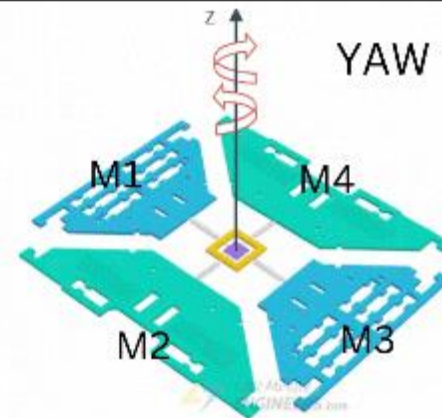


Fig.3

TCS34725 (RGB Color)

Measures Red, Green, Blue and Clear light channels via I2C.

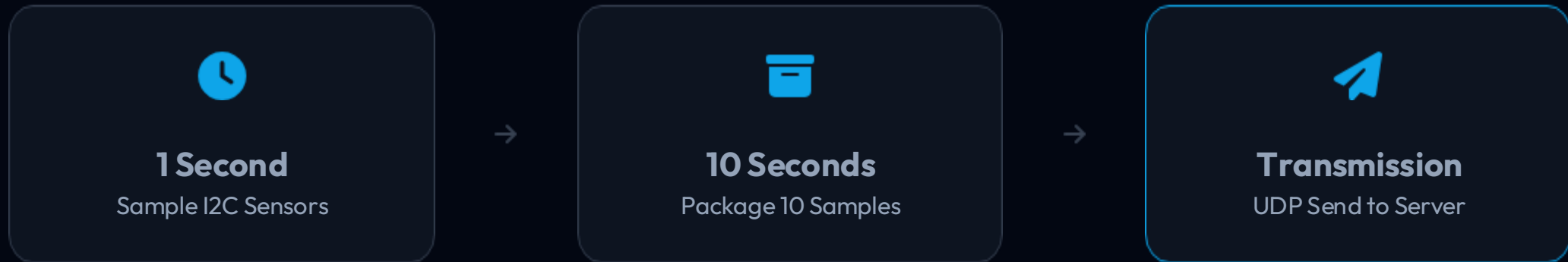
MPU6050 (Accel/Gyro)

Captures 3-axis acceleration and rotation metrics.

Sampling Rate: 1 Sample per second.

Buffer Size: 10 samples before UDP transmission.

Timing & Coordination



Data Formatting Strategy: CSV/String serialization used to ensure the server parses Acceleration (3 axes) and RGB values without data collision.

Real-Time Statistical Engine

The Ubuntu Server computes essential metrics for every batch of data received:

Mean

μ

Max/Min

↑↓

Std Deviation

$$\sigma = \sqrt{[\sum (x - \mu)^2 / N]}$$

```
--- PACHET 10 SECUNDE PRIMIT. STATISTICI: ---
```

```
Rosu      - Mean: 810.90 | Min:    0 | Max:   907 | StdDev: 270.36
Verde     - Mean: 908.50 | Min:    0 | Max:  1017 | StdDev: 302.91
Albastru  - Mean: 879.00 | Min:    0 | Max:   985 | StdDev: 293.09
Accel X   - Mean:  -0.78 | Min: -0.87 | Max:   0.00 | StdDev:  0.26
Accel Y   - Mean:  -0.32 | Min: -0.36 | Max:   0.00 | StdDev:  0.11
Accel Z   - Mean:   0.06 | Min:  0.00 | Max:   0.07 | StdDev:  0.02
```

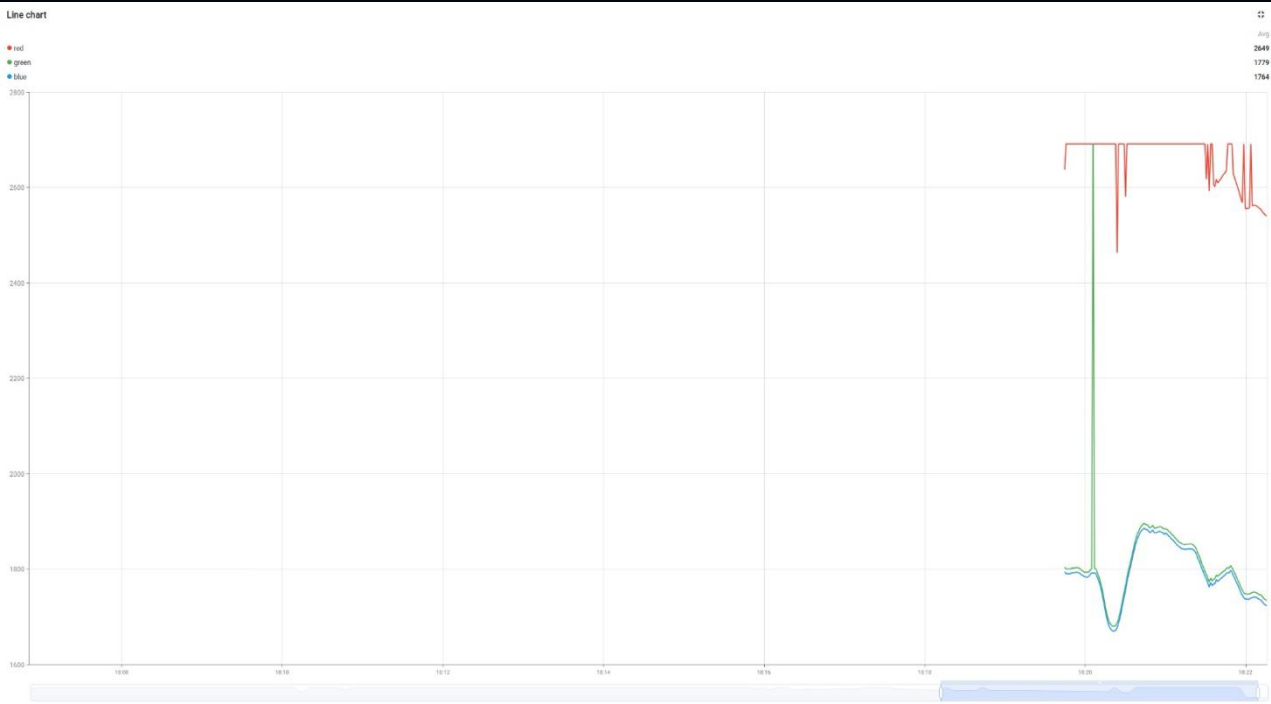
MQTT & ThingsBoard

Integration

ThingsBoard Implementation

- MQTT Client on RPi (mosquitto_pub)
- Call system() within C code for telemetry
- Live Dashboards for RGB & Motion data

```
mosquitto_pub -h "hostip" -t "v1/devices/me/telemetry" -m '{"accel":0.8}'
```



Time series table

Realtime - last 1 hour

Timestamp ↓	accel_x	accel_y	accel_z	blue	green	red
2026-05-19 18:20:10	-0.01	-0.01	0.99	1775	1784	2691
2026-05-19 18:20:09	-0.01	-0.01	0.99	1782	1792	2691
2026-05-19 18:20:08	-0.01	-0.01	0.99	1790	1800	2691
2026-05-19 18:20:07	-0.01	-0.01	0.99	1792	1802	2691
2026-05-19 18:20:05	-0.01	-0.01	0.99	1791	2691	2691
2026-05-19 18:20:04	-0.01	-0.01	0.99	1792	1801	2691
2026-05-19 18:20:03	-0.01	-0.01	0.99	1789	1798	2691
2026-05-19 18:20:02	-0.01	-0.01	0.99	1785	1794	2691
2026-05-19 18:20:01	-0.01	-0.01	0.99	1783	1793	2691
2026-05-19 18:20:00	-0.01	-0.01	0.99	1783	1793	2691

Project Fulfillment & Demo

Mandatory Deliverables

- Fully commented C/C++ Source Code
- Makefile for automated compilation
- Logbook (detailed record of dev steps)
- Buildroot Linux configuration

[Play Demo](#) 

Final Technical Conclusions



Mastery

Acquired deep understanding of Embedded Linux, Buildroot, and cross-compilation.



Interoperability

Proved the efficiency of UDP for real-time sensing vs MQTT for cloud persistence.



Ready for IoT

System demonstrates a complete professional path from raw bits to global UI.

Questions?

Thank you for your attention.

Image Sources



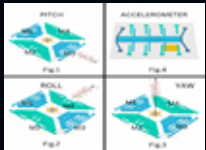
https://miro.medium.com/1*lw5ULtUQBmP_CY8aJbKBxg.png

Source: [medium.com](https://miro.medium.com/1*lw5ULtUQBmP_CY8aJbKBxg.png)



https://miro.medium.com/v2/resize:fit:2000/1*46JX-o6xxMaMmOmTLwyLYQ.png

Source: [medium.com](https://miro.medium.com/v2/resize:fit:2000/1*46JX-o6xxMaMmOmTLwyLYQ.png)



<https://europe1.discourse-cdn.com/arduino/original/4X/2/1/d/21de57f5a75b9d10a7513db8d07404882b294fc6.gif>

Source: [forum.arduino.cc](https://europe1.discourse-cdn.com/arduino/original/4X/2/1/d/21de57f5a75b9d10a7513db8d07404882b294fc6.gif)



<https://norvi.io/wp-content/uploads/2025/11/connecting-esp32-to-thingsboard-over-wi-fi-img-create-a-dashboard-for-visualization-02.png>

Source: [norvi.io](https://norvi.io/wp-content/uploads/2025/11/connecting-esp32-to-thingsboard-over-wi-fi-img-create-a-dashboard-for-visualization-02.png)